Liver Patient Classification Using ML Methods

Lucas Viner*, Oriol Parent*
*GCED - Universitat Politècnica de Catalunya, Barcelona

***Abstract*—Aquest treball descriu el desenvolupament d'un sistema de classificació per a la detecció de malalties hepàtiques utilitzant dades clíniques i demogràfiques. El conjunt de dades, provinent del repositori UCI ML, presenta un fort desequilibri de classes, fet que ha condicionat tant la selecció de mètriques com l'estratègia de modelatge. S'han aplicat transformacions per normalitzar les distribucions i optimització exhaustiva d'hiperparàmetres. Es comparen múltiples models supervisats mitjançant validació creuada estratificada, amb l'F1-score com a mètrica principal. Els resultats evidencien el potencial de l'aprenentatge automàtic per millorar la detecció precoç en entorns clínics.**

***Index Terms*—Machine Learning, Liver Disease, Classification, F1-score, Class Imbalance.**

I. Introducció

Aquest treball aborda la classificació de pacients amb possibles malalties hepàtiques mitjançant tècniques d'aprenentatge automàtic, en el marc d'una competició a la plataforma *Kaggle InClass*. A partir d'un conjunt de dades extret del repositori *UCI Machine Learning Repository*, que conté informació clínica i demogràfica de pacients, es busca construir un model predictiu robust capaç de determinar si un pacient presenta una malaltia hepàtica o no.

L'enfocament segueix el cicle complet de desenvolupament d'un sistema de classificació: anàlisi exploratòria de dades, tractament de valors atípics, transformacions estadístiques, enginyeria de variables, selecció i optimització d'algoritmes supervisats, així com l'ús de mètodes d'ensembles per millorar la capacitat predictiva. El desequilibri entre classes ha estat abordat tant a nivell de mostreig com amb ajustos en la funció de pèrdua dels models.

En aquest informe es presenten, de manera estructurada, els passos seguits per a la preparació del conjunt de dades, la selecció de les variables més rellevants, els criteris d'avaluació utilitzats (amb èmfasi en l'F1-score), els resultats dels models més destacats i les conclusions obtingudes, tot proposant línies de millora futures.

II. Càrrega de dades i preprocessat

A. Càrrega i divisió de les dades

Inicialment, es carreguen les dades i es divideixen en dos subconjunts: conjunt d'entrenament (*train, 80%*) i conjunt de validació externa (*validation, 20%*). Aquesta partició permet avaluar el rendiment dels models de manera objectiva i detectar possibles fenòmens de sobreajust. No ha calgut separar un conjunt de test addicional, ja que aquest ja era proporcionat per la competició *InClass*.

B. Estudi del preprocessat

El procés de preprocessat s'ha estructurat en dues etapes:

- **Anàlisi exploratòria:** s'examina la naturalesa de les dades per comprendre les característiques generals de les dades.
- **Transformacions:** es realitzen les transformacions identificades com a necessàries en l'anàlisi exploratòria per estabilitzar la variància, millorar la normalitat i preparar-les per l'entrenament de models.

1) Anàlisi de valors nuls i outliers: Rere una breu anàlisi del dataset i no trobar anomalies a simple vista, comencem per verificar que les dades no contenen valors nuls, tal com ja s'havia indicat prèviament. A continuació, s'analitzen els possibles *outliers*. Tot i detectar-ne presència en diverses variables, decidim no eliminar-los. El motiu principal és el context mèdic del problema, on cada observació pot tenir una rellevància clínica significativa. A més, eliminar aquests valors va empitjorar el rendiment del model (F1-score més baix), fet que reforça la decisió de mantenir-los.

2) Anàlisi de gaussianitat: També s'analitza la distribució de les variables per tal d'avaluar la seva gaussianitat. La majoria de variables presenten una distribució allunyada d'una normal, excepte `TP`, `ALB` i `A/R`, que mostren un comportament més proper a una distribució normal. Per tant decidim aplicar el logaritme a totes les variables que hem vist que no segueixen normalitat.

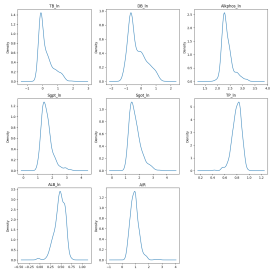


Fig. 1. Distribucions de densitat estimades per a les variables contínues del conjunt de dades després d'aplicar la transformació logarítmica.

3) Anàlisi de rangs i col·linealitat: Amb l'objectiu de detectar possibles problemes de col·linealitat entre variables, es calcula en primer lloc la matriu de correlació. Aquesta anàlisi, però, pot veure's distorsionada si les variables tenen escales molt diferents. Per aquest motiu, es duu a terme un estudi previ dels rangs de les variables mitjançant la funció `describe`.

S'observa que la majoria de variables tenen valors en un rang similar, degut a que ja se'ls havia aplicat una transformació logarítmica. No obstant això, tres variables (`TP`,

ALB i A/R) conserven valors molt més propers a l'escala original. Per garantir una comparabilitat adequada i evitar biaixos en l'estimació de correlacions, es decideix aplicar també la transformació logarítmica a aquestes tres variables.

Un cop fet aquest ajust, es torna a calcular la matriu de correlació.

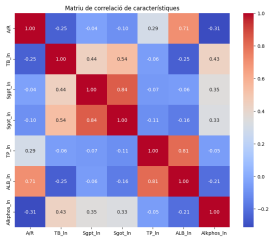


Fig. 2. Heatmap de la matriu de correlació entre les variables després de la transformació logarítmica. Es pot observar una col·linealitat elevada entre TB_ln i DB_ln.

Es detecta una col·linealitat elevada entre TB i DB, dues mesures relacionades amb la bilirrubina. S'exploren diverses estratègies, com la creació d'una nova variable basada en el seu quocient, però aquesta no aporta millores en el rendiment del model. Finalment, es decideix eliminar DB i mantenir TB_ln com a variable representativa.

4) Desbalanceig de classes: S'ha intentat equilibrar el dataset desbalancejat aplicant la tècnica SMOTE, però no s'han obtingut millores en el rendiment del model. Això suggereix que, en aquest cas, SMOTE no és una estratègia efectiva.

C. Construcció de la funció

Un cop completada l'anàlisi exploratòria i definides les transformacions a aplicar, es construeix una *funció* de preprocessat que encapsula tot el procés de transformació de les dades: preprocess_features. Aquesta *funció* aplica de manera sistemàtica la transformació logarítmica a totes les variables rellevants i elimina la variable DB, seguint les conclusions obtingudes en les seccions anteriors. L'ús d'una funció dintre d'un *pipeline* permet garantir un flux de treball coherent, segur i reproduïble, assegurant que el preprocessat es manté consistent tant durant la fase d'entrenament com en la de validació o inferència.

III. Models de classificació

A. Models Simples

Aquesta secció presenta l'avaluació de diversos models de classificació supervisada estudiats amb l'objectiu d'identificar aquells que ofereixen millor capacitat de classificació.

Abans de construir el *pipeline* i realitzar la cerca exhaustiva d'hiperparàmetres per seleccionar el millor model, es va dur a terme una primera avaluació de diversos models de classificació. Aquesta etapa exploratòria tenia com a objectiu comparar el rendiment bàsic de diferents algoritmes emprant els seus paràmetres per defecte o mínimament ajustats, amb l'objectiu d'identificar quins podien oferir més potencial per a l'optimització posterior.

A continuació es presenten els models avaluats, juntament amb una breu explicació del seu funcionament:

- **Logistic Regression**: Model lineal que estima la probabilitat de pertinença a una classe mitjançant una funció sigmoide. És senzill, interpretable i sovint utilitzat com a base comparativa.
- **Logistic Regression (L1)**: Variant de regressió logística amb regularització L1 (*Lasso*), que pot forçar alguns coeficients a zero, afavorint així la selecció de variables.
- **Support Vector Machine (SVM)**: Algoritme que busca el millor hiperplà que separa les classes amb el màxim marge. Funciona bé en espais d'alta dimensionalitat.
- **Support Vector Machine (SVM RBF)**: Versió no lineal de l'SVM que utilitza un nucli de funció radial (*RBF*) per adaptar-se a fronteres de decisió no lineals.
- **Random Forest**: Conjunt d'arbres de decisió entrenats sobre mostres aleatòries. Redueix el sobreajustament i ofereix bones prestacions generals.
- **Decision Tree**: Algoritme senzill que construeix un arbre per dividir l'espai de dades. És fàcil d'interpretar, però propens al sobreajustament.
- **Extra Trees**: Variant del Random Forest amb més aleatorietat en la construcció dels arbres. Pot reduir la variància del model i millorar la seva capacitat de generalització.
- **K-Nearest Neighbors (KNN)**: Classificador basat en la distància. Classifica segons la majoria de veïns més propers. És senzill però pot ser ineficient amb grans conjunts de dades.
- **Linear Discriminant Analysis (LDA)**: Model lineal que assumeix que les classes tenen distribució normal amb la mateixa matriu de covariància. Projecta les dades per maximitzar la separació entre classes.
- **Quadratic Discriminant Analysis (QDA)**: Similar a LDA, però permet que cada classe tingui una matriu de covariància pròpia. Més flexible però pot sobreajustar si hi ha poques dades.
- **Naive Bayes**: Model probabilístic basat en el teorema de Bayes, amb l'assumpció d'independència entre variables. Ràpid, eficient i sorprenentment competitiu en molts escenaris.

Aquesta comparació inicial va permetre identificar els models amb un comportament més prometedor, concretament *Random Forest*, *Logistic Regression* i *SVM*. Tot i així, com que les diferències de rendiment entre els classificadors no eren substancials, es va optar per dur a terme una optimització exhaustiva d'hiperparàmetres per a cadascun dels models.

B. Models Ensamble

Amb l'objectiu de millorar el rendiment obtingut pels models individuals, s'han explorat diverses tècniques d'ensamblatge, incloent-hi els mètodes de *Voting Classifier* tant en la seva modalitat *hard* (majoria simple) com *soft* (promig de probabilitats), així com estratègies de *Stacking*, que combinen diversos models base mitjançant un model meta-final.

Tanmateix, no només es van construir aquests ensamblatges de forma estàtica, sinó que es va dur a terme una optimització exhaustiva dels seus paràmetres mitjançant recerques en reixeta (*GridSearchCV*). En el cas del *VotingClassifier*, es van avaluar combinacions de pesos entre els models base per a la modalitat *soft voting*, amb diferents valors possibles (`[0.5, 1.0, 1.5, 2.0]`) per maximitzar el *F1-score macro* com a mètrica objectiu. Per als mètodes de *stacking* i altres ensembles, també es van explorar diverses configuracions d'hiperparàmetres, ajustant-se als millors resultats obtinguts durant la validació creuada.

Tot i aquest esforç d'ajust fi dels ensamblatges, cap de les combinacions avaluades va aconseguir superar els resultats dels millors models simples seleccionats en la fase de cerca d'hiperparàmetres. Per tant, es va optar per mantenir aquests models bàsics com a solució principal, prioritzant la simplicitat, la interpretabilitat i l'eficiència computacional.

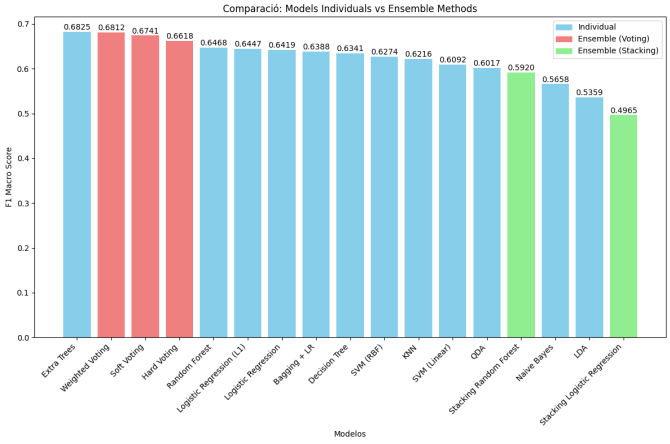


Fig. 3. Comparació de rendiment: models individuals vs. ensemble

IV. Mètriques d'avaluació

Per tal d'entendre millor com s'han avaluat els models anteriors, cal aprofundir en el significat de cadascuna de les mètriques utilitzades. Això és especialment important en un context de desequilibri de classes, on certes mètriques poden resultar enganyoses. En aquest treball s'han emprat diverses mètriques, tot centrant-nos principalment en el **F1-score macro**. A continuació es detallen les mètriques considerades i la justificació de la seva selecció.

Accuracy: L'*accuracy* mesura el percentatge de prediccions correctes sobre el total:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

on TP, TN, FP i FN són vertaders positius, vertaders negatius, falsos positius i falsos negatius, respectivament. Tot i ser comuna, l'*accuracy* pot ser enganyosa en casos de classes desequilibrades, ja que pot afavorir la classe majoritària.

F1-score: El *F1-score* és una mètrica basada en la mitjana harmònica de la *precisió* (precision) i la *recuperació* (recall). Aquesta mètrica és especialment útil quan existeix un desequilibri entre classes, ja que penalitza més fortament els errors que afecten la classe minoritària. Les fórmules són les següents:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{2}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{3}$$

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4}$$

En aquest treball s'han calculat diverses variants:

- **F1-score per la classe positiva (1)**: reflecteix el rendiment del model a l'hora de detectar la classe positiva, sovint la més rellevant clínicament o operativament.
- **F1-score per la classe negativa (0)**: també és útil per analitzar si el model ignora totalment la classe majoritària o aconsegueix mantenir un equilibri.
- **F1-score macro**: mitjana aritmètica dels F1-score de cada classe, sense tenir en compte la proporció d'aparició de cadascuna. Aquesta mètrica s'ha emprat com a criteri principal d'optimització (`refit`) dins de la validació creuada per garantir un tracte equitatiu a ambdues classes.

$$F1_{\text{macro}} = \frac{F1_0 + F1_1}{2} \tag{5}$$

Justificació de la tria: Donat que les dades presenten un cert grau de desequilibri entre les dues classes, basar-nos únicament en la *accuracy* podria conduir a una falsa sensació de bon rendiment. Per aquest motiu, es prioritza el **F1-score macro**, ja que:

1) Considera de forma igualitària ambdues classes, independentment de la seva freqüència.
2) Penalitza fortament els models que només s'ajusten bé a una de les dues classes.
3) Ofereix una visió més realista i robusta del comportament general del model en entorns on els errors poden tenir diferent impacte (per exemple, falsos positius vs. falsos negatius).

A més, la disponibilitat de totes les mètriques (accuracy, F1 per classe i F1 macro) ha permès fer una anàlisi detallada posterior del comportament de cada model i justificar millor la tria del classificador final.

V. Experiments

A. Cerca d'hiperparàmetres i selecció del millor model

Per tal de garantir una comparació justa i sistemàtica, s'ha construït una infraestructura modular basada en *pipelines* escalables, que permet aplicar de manera homogènia les transformacions necessàries, ajustar els hiperparàmetres i avaluar el rendiment mitjançant validació creuada.

La cerca d'hiperparàmetres s'ha realitzat amb la tècnica de `GridSearchCV`.

- *Construcció del pipeline:* Cada *pipeline* consta de quatre etapes:

1) **Preprocessat**: s'utilitza un `FunctionTransformer` per aplicar la funció `preprocess_features` dins del *pipeline*. Aquesta funció ha estat prèviament definida i s'encarrega d'aplicar el logaritme a totes les variables numèriques, eliminar la variable `DB` i dur a terme altres transformacions identificades com a rellevants durant la fase d'anàlisi exploratòria. La integració d'aquesta funció en el *pipeline* permet garantir consistència i evitar filtracions d'informació durant la validació creuada.
2) **Escalat de les variables**: segons la naturalesa del model utilitzat, s'aplica un escalat mitjançant `StandardScaler`, `RobustScaler` o `MinMaxScaler`, o bé es manté sense escalar (`passthrough`). Aquesta etapa és especialment crítica en models sensibles a les distàncies, com SVM o KNN.
3) **Reducció de dimensionalitat (PCA)**: es considera l'ús opcional de `PCA` per a reduir la dimensionalitat. En els casos en què s'utilitza, es defineix la proporció de variància a preservar com a hiperparàmetre (`n_components`), normalment entorn del 68–93%.
4) **Model**: s'insereix el classificador corresponent. Per a cada model, s'ha definit un conjunt d'hiperparàmetres específics, incloent opcionalment `class_weight` per abordar el desequilibri, així com paràmetres propis com `C`, `gamma`, `penalty`, etc.

- *Definició dels models i espai de cerca:* Amb la infraestructura dels *pipelines* establerta, es van definir diversos models base de classificació, cadascun amb el seu conjunt d'hiperparàmetres a explorar.

Per tal de realitzar la cerca d'hiperparàmetres, s'ha especificat per a cada model una graella que inclou:

- el tipus d'escalador (`StandardScaler`, `RobustScaler`, `MinMaxScaler` o cap),
- la presència o absència de reducció de dimensionalitat amb `PCA`,
- i els hiperparàmetres específics del model (`C`, `gamma`, `n_neighbors`, `max_depth`, etc.).

També s'ha considerat, sempre que sigui compatible amb el model, la inclusió del paràmetre `class_weight` amb valors com `balanced` o `balanced_subsample`, amb l'objectiu d'atenuar l'impacte del desbalanceig entre classes.

El procés de cerca s'ha dut a terme utilitzant `GridSearchCV`, amb una validació creuada estratificada i repetida mitjançant `RepeatedStratifiedKFold` ($k = 5$, $r = 10$), garantint així estabilitat i robustesa en l'avaluació. La mètrica escollida per optimitzar ha estat l'`f1_macro`, que dona el mateix pes a cada classe, fet especialment rellevant en entorns clínics amb baixa representació d'algunes categories.

Per a cada model, s'ha registrat el millor estimador trobat, la puntuació corresponent i els resultats de totes les combinacions avaluades, permetent així comparar de forma objectiva el rendiment global i seleccionar el model òptim per al problema tractat.

- *Avaluació dels models:* Tota la cerca s'ha dut a terme mitjançant `GridSearchCV`, utilitzant validació creuada estratificada i repetida (`RepeatedStratifiedKFold`) amb 5 particions i 10 repeticions. Aquest enfocament ofereix una estimació estable i robusta del rendiment mitjà dels models, especialment rellevant en contextos amb conjunts de dades petits i desequilibrats, com és habitual en entorns biomèdics.

A més, l'estratègia s'ha dissenyat per ser reproductible, permetent futures iteracions o adaptacions a altres conjunts de dades sense redefinir tot el procés. Com a mètrica principal per seleccionar el millor model s'ha utilitzat el **F1-score macro**, tot i que també s'han registrat els valors de `accuracy` i `F1-score` per classe (pos_label = 1 i 0) per dur a terme una anàlisi més fina i equilibrada del comportament de cada model.

Els resultats obtinguts s'han estructurat inicialment en un diccionari, posteriorment convertit en un `DataFrame`, i ordenat segons el millor `F1-score macro`. Aquesta taula ha servit com a base objectiva per identificar el model simple més competitiu, així com els hiperparàmetres òptims associats, garantint una selecció informada i quantitativament fonamentada.

VI. Resultats

Un cop completada la cerca d'hiperparàmetres i l'ajust dels models mitjançant validació creuada, es procedeix a l'anàlisi detallat dels resultats obtinguts.

Els resultats mostren que els models basats en arbres, especialment l'Extra Trees i el Random Forest, obtenen els millors resultats de classificació a nivell de models simples, amb una puntuació de 0.682 i 0.647 respectivament. Això indica que aquests models són especialment efectius per capturar patrons complexos en les dades.

Els models lineals com la regressió logística (tant amb L1 com sense) també mostren un rendiment raonable (al voltant de 0.64), tot i ser més senzills i interpretables. Els mètodes de màquina de vector suport (SVM) i KNN es situen a la franja mitjana, mentre que els models més bàsics com el Naive Bayes i el LDA tenen un rendiment clarament inferior, probablement degut a les seves hipòtesis més restrictives sobre les dades.

Aquest comportament suggereix que el problema presenta certa complexitat no lineal i que la capacitat dels models per manejar interaccions entre variables és clau. Cal destacar que, malgrat els esforços d'optimització, el millor mètode d'ensamblatge no ha aconseguit superar el model Extra Trees, tot i que s'hi ha acostat amb una puntuació molt propera. Això reforça la idea que, en aquest cas, un bon model individual pot oferir un rendiment equiparable o superior a combinacions més complexes.

TABLE I
F1-SCORE I HIPERPARÀMETRES COMPLETS DELS MILLORS MODELS

Model	F1 i Paràmetres Complets
Extra Trees	0.682 – class_weight='balanced', max_depth=10, max_features=0.45, min_samples_leaf=3, min_samples_split=2, n_estimators=310, pca='passthrough', scaler='passthrough'
Random Forest	0.647 – class_weight='balanced_subsample', max_depth=3, max_features='sqrt', min_samples_leaf=10, min_samples_split=2, n_estimators=196, pca='passthrough', scaler='passthrough'
LogReg (L1)	0.645 – C=0.15, class_weight='balanced', pca='passthrough', scaler=RobustScaler()
Logistic Reg.	0.642 – C=0.18, class_weight='balanced', solver='liblinear', pca=PCA(n_components=0.685), scaler=StandardScaler()
Bagging + LR	0.639 – estimator_C=1.45, estimator_penalty='l1', estimator_solver='saga', max_features=0.425, max_samples=0.73, n_estimators=36, pca=PCA(n_components=0.685), scaler=StandardScaler()
Decision Tree	0.634 – class_weight='balanced', max_depth=2, min_samples_leaf=13, min_samples_split=2, pca='passthrough', scaler='passthrough'
SVM (RBF)	0.627 – C=79, class_weight='balanced', gamma='scale', kernel='rbf', pca=PCA(n_components=0.69), scaler=MinMaxScaler()
KNN	0.622 – leaf_size=1, metric='euclidean', n_neighbors=5, weights='distance', pca=PCA(n_components=0.935), scaler=StandardScaler()
SVM (Linear)	0.609 – C=1.45, class_weight='balanced', kernel='linear', pca=PCA(n_components=0.685), scaler=StandardScaler()
QDA	0.602 – priors=None, reg_param=1.0, store_covariance=True, tol=0.0001, pca='passthrough', scaler=StandardScaler()
Naive Bayes	0.566 – priors=None, var_smoothing=5e-10, pca='passthrough', scaler=StandardScaler()
LDA	0.536 – n_components=None, priors=None, shrinkage='auto', solver='lsqr', store_covariance=False, tol=0.0001, pca='passthrough', scaler=StandardScaler()

Passem a comentar els millors hiperparàmetres obtinguts.

A. Anàlisi dels millors hiperparàmetres per famílies de models

A partir de l'anàlisi dels millors resultats per model, podem extreure les següents observacions, agrupades per tipus de model:

Models basats en arbres:

- **Extra Trees** és el model amb millor rendiment (*F1* = 0,6825), amb poda moderada (`max_depth` = 10) i restriccions com `min_samples_leaf` = 3. L'ús de `passthrough` suggereix que les dades originals ja són adequades.
- **Random Forest** obté un bon resultat (*F1* = 0,6468) amb baixa profunditat (`max_depth` = 3) i ponderació `balanced_subsample`.
- **Decision Tree** assoleix el millor rendiment amb un arbre molt simple (`max_depth` = 2), mostrant la importància del control de complexitat.

Models lineals:

- **Logistic Regression** (L1 i clàssica) mostra rendiment robust (*F1* 0,641–0,644), amb regularització forta i bon escalat.
- **Bagging + Logistic** millora lleugerament la robustesa, però no supera les versions simples.

Models basats en distància:

- **SVM (RBF)** requereix `C` alt (79), gamma suau i `MinMaxScaler`, però no supera models més senzills.
- **SVM (Linear)** utilitza `C` = 1.45 i `StandardScaler`, amb rendiment lleugerament inferior al RBF.
- **KNN** funciona millor amb mètrica `euclidean`, pocs veïns (`n_neighbors` = 5) i `PCA` amb 93.5% de variància.

Models bayesians i discriminants:

- **QDA** i **LDA** milloren amb ajustos de `reg_param`, `tol` i `shrinkage`, però tenen rendiment inferior als arbres i la regressió logística.
- **Naive Bayes** funciona millor amb `var_smoothing` molt baix (`5e-10`), però obté el *F1* més baix degut a les seves fortes assumpcions.

Conclusions generals: Els millors resultats es donen en models basats en arbres amb poda acurada i mínimes transformacions. Els models lineals s'hi acostin amb bon ajust i escalat adequat. L'ús de `PCA` i escalat millora clarament models de distància com `KNN` i `SVM`. Els models probabilístics i discriminants, malgrat un ajust fi, presenten limitacions amb dades complexes.

VII. CONCLUSIONS

El present treball ha evidenciat l'eficàcia de l'aprenentatge automàtic com a eina per a la detecció i classificació de malalties hepàtiques a partir de dades clíniques i demogràfiques. El conjunt de dades utilitzat, obtingut del repositori UCI, presentava un marcat desequilibri entre classes, fet que ha condicionat significativament les estratègies de modelatge, la selecció de mètriques d'avaluació i la configuració dels classificadors.

S'ha dut a terme un procés rigorós de preprocessat que ha inclòs l'anàlisi de valors atípics, la transformació de variables no normalment distribuïdes mitjançant funcions logarítmiques, i la detecció de col·linealitat entre atributs. Aquest enfocament ha permès preservar la màxima informació rellevant, especialment en un context clínic on cada dada pot tenir una interpretació mèdica significativa. L'ús d'un pipeline estructurat ha garantit la coherència del procés entre les fases d'entrenament i validació, contribuint a la robustesa del sistema desenvolupat.

Pel que fa als models, s'ha fet una avaluació exhaustiva de classificadors simples i ensamblats, considerant una àmplia gamma d'estratègies, des de mètodes lineals interpretables com la regressió logística, fins a tècniques complexes com els boscos aleatoris i el gradient boosting. Els resultats han posat de manifest que els models basats en arbres, especialment l'Extra Trees Classifier, ofereixen un rendiment superior, capturant millor les relacions no lineals entre variables i mostrant una elevada capacitat predictiva en presència de dades desequilibrades.

En definitiva, aquest treball consolida la viabilitat de construir sistemes predictius fiables aplicats a dades mèdiques reals, destacant la importància de combinar una bona anàlisi estadística prèvia, una selecció acurada d'algoritmes i una validació creuada rigorosa per assolir resultats robustos i clínicament útils.

REFERENCES

[1] J. Vidal, *Apunts i presentacions de l'assignatura*, Aprenentatge Automàtic 1, Universitat Politècnica de Catalunya, 2024–2025. Materials disponibles a través d'Atenea.

[2] Alexis Molina Martinez De Los Reyes, *Pràctiques de laboratori de l'assignatura Aprenentatge Automàtic 1*, Universitat Politècnica de Catalunya, 2024–2025.